

Software Engineering

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Software Engineering — Complete Notes	4
1. Introduction to Software Engineering	4
1.1 Software and Software Engineering	4
1.2 Software Characteristics	4
1.3 Software Components and Quality Attributes	5
1.4 Software Crisis	5
1.5 Software Myths	5
1.6 Software Process and Process Framework	5
2. Software Development Life Cycle (SDLC)	6
2.1 SDLC — Definition and Importance	6
2.2 Software Configuration Management (SCM)	6
3. Software Development / Process Models	7
3.1 Waterfall Model (Classical Linear Sequential)	7
3.2 Iterative / Incremental Model	7
3.3 Prototyping Model	8
3.4 Spiral Model (Risk-Driven)	8
3.5 RAD Model (Rapid Application Development)	8
3.6 V-Model (Verification and Validation Model)	9
3.7 Agile Model (Scrum, XP, Kanban)	9
3.8 Comparison and Selection of Models	10
4. Requirement Engineering	10
4.1 Definition and Tasks	10
4.2 Requirement Elicitation Techniques	10
4.3 Types of Requirements: Functional vs Non-Functional	11
4.4 Requirement Analysis and Modeling	11
4.5 Software Requirements Specification (SRS)	11
4.6 Requirement Validation and Management	11
4.7 Feasibility Study	11
5. Software Design	12
5.1 Design Concepts and Principles	12
5.2 Cohesion (Intra-Module — HIGH is Good)	12
5.3 Coupling (Inter-Module — LOW is Good)	13
5.4 Design Approaches	13
6. Software Metrics and Project Management	14
6.1 Software Measurement and Metrics	14
6.2 COCOMO — Constructive Cost Model	15

6.3 Project Scheduling (Gantt, PERT/CPM)	15
6.4 Risk Management	16
6.5 Software Quality Assurance (SQA)	16
7. Software Testing	16
7.1 Testing Fundamentals and Objectives	16
7.2 Levels of Testing (Order)	17
7.3 Integration Testing Strategies — Stubs vs Drivers	17
7.4 White-Box (Structural) Testing	18
7.5 Black-Box (Functional) Testing	18
7.6 Static Testing Strategies (No Code Execution)	19
8. Software Maintenance	20
8.1 Need and Process	20
8.2 Types of Maintenance (4)	20
8.3 Re-engineering and Reverse Engineering	20
8.4 Software Reliability and Availability	20
9. Quick Revision / Exam Pointers	21

Software Engineering — Complete Notes

1. Introduction to Software Engineering

1.1 Software and Software Engineering

Definition of Software

Software is a collection of **computer programs**, associated **documentation**, and **operating procedures/configuration data** that together perform a desired function or task.

The classic equation is: **Software = Programs + Documentation + Operating Procedures**. A program alone is NOT software until it is documented and packaged for use.

Software is **logical** (intellectual) rather than **physical**, so its cost is concentrated in **engineering/design**, not in **manufacturing**.

Definition of Software Engineering

The **IEEE** definition: Software Engineering is "the application of a **systematic, disciplined, quantifiable** approach to the **development, operation and maintenance** of software; that is, the application of **engineering** to software."

Fritz Bauer (1968, NATO conference) gave the first definition: "establishment and use of **sound engineering principles** to obtain **economically** software that is **reliable** and works **efficiently** on real machines."

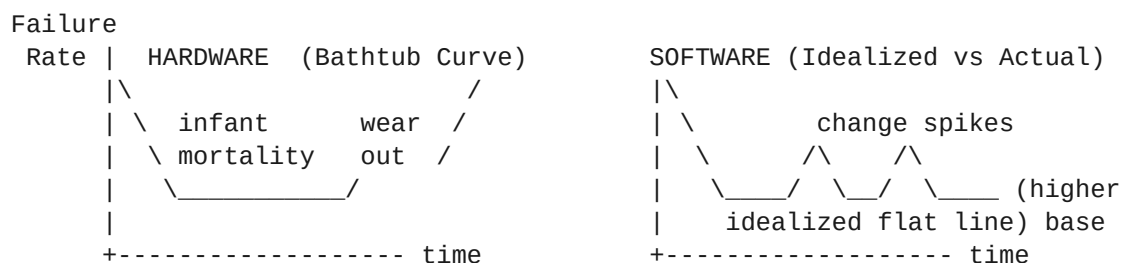
Memory Trick: Bauer's three goals = **Economical, Reliable, Efficient** → "**ERE**". The IEEE keyword trio = **Systematic, Disciplined, Quantifiable**.

1.2 Software Characteristics

The **three** key characteristics (Pressman) that distinguish software from hardware:

- **Software is developed/engineered, it is NOT manufactured** — there is no manufacturing phase that introduces quality problems; quality is built during design.
- **Software does NOT wear out, but it deteriorates** — hardware wears out due to environmental effects; software fails due to undiscovered defects and degrades because each **change** introduces new defects.
- **Most software is custom-built, not assembled from existing components** — though component reuse is increasing.

The Software Failure Curve (Bathtub vs Software)



Hardware follows the **bathtub curve** (high **infant-mortality** failures, a flat middle, then a rising **wear-out** region). Software has **no wear-out**; instead each maintenance **change** causes a **spike**, and the curve settles at a **higher** failure rate than before — this is **software deterioration**.

MCQ Trap: "Software **wears out**" is FALSE. Software **deteriorates** due to change. Only **hardware** wears out.

1.3 Software Components and Quality Attributes

Software Quality Models

McCall's quality model groups **11** quality factors into **3** categories:

Category	Quality Factors
Product Operation	Correctness, Reliability, Efficiency, Integrity, Usability
Product Revision	Maintainability, Flexibility, Testability
Product Transition	Portability, Reusability, Interoperability

ISO 9126 defines **6** quality characteristics: **Functionality, Reliability, Usability, Efficiency, Maintainability, Portability**.

FURPS (Hewlett-Packard) = **Functionality, Usability, Reliability, Performance, Supportability**.

Memory Trick: ISO 9126 six = "**FRUEMP**" (Functionality, Reliability, Usability, Efficiency, Maintainability, Portability).

1.4 Software Crisis

The **Software Crisis** refers to the set of problems faced in the 1960s–70s when software projects routinely ran **over budget, over schedule**, were of **poor quality, hard to maintain**, and **failed to meet requirements**.

The term was popularised at the **1968 NATO Software Engineering Conference** (Garmisch, Germany), which also coined the discipline name "**Software Engineering**".

Causes: rising **hardware power** and **system complexity**, larger problem sizes, and the absence of disciplined methods.

1.5 Software Myths

Beliefs that appear true but cause project failure. Pressman classifies them into **3** groups:

- **Management myths** — e.g., **"We have a book of standards, so that's enough"**; **"If we fall behind, we can add more programmers"** (refuted by **Brooks's Law**); **"If I outsource it, I can relax."**
- **Customer myths** — e.g., **"A general statement of objectives is enough to start coding; details can come later"**; **"Requirements change, but change is easy to accommodate because software is flexible."**
- **Practitioner/Developer myths** — e.g., **"Once the program works, the job is done"**; **"The only deliverable is the working program"**; **"Until the program runs, quality cannot be assessed."**

MCQ Trap: Brooks's Law = **"Adding manpower to a late software project makes it later."** (from **"The Mythical Man-Month"**, Fred Brooks, 1975). It refutes the **management** myth, not a customer myth.

1.6 Software Process and Process Framework

A **Software Process** is a **framework** for the **activities, actions and tasks** required to build high-quality software.

Pressman's Generic Process Framework has **5** framework activities: **Communication, Planning, Modeling, Construction, Deployment**.

Umbrella activities run across the entire process: **Project tracking & control, Risk management, SQA, Technical reviews, Measurement, SCM (Configuration Management), Reusability management, Work-product preparation**.

Memory Trick: 5 framework activities = "**C-P-M-C-D**" (Communication → Planning → Modeling → Construction → Deployment).

2. Software Development Life Cycle (SDLC)

2.1 SDLC — Definition and Importance

The **Software Development Life Cycle (SDLC)** is a **structured sequence of well-defined phases** used to develop, deliver and maintain software, ensuring **quality, predictability and control**.

Phases of SDLC (in order)

The standard **7** phases, in order, are:

Planning/
Feasibility --> Requirement
Analysis --> Design
(HLD+LLD) --> Coding/
Implementation --> Testing
(V&V) --> Deployment
(Release)

- **1. Planning / Feasibility Study** — define scope, cost, schedule; check technical/economic/operational feasibility.
- **2. Requirement Analysis** — gather and document **what** the system must do; produces the **SRS**.
- **3. Design** — define **how**; architecture, modules, interfaces (**HLD** = High-Level Design, **LLD** = Low-Level Design).
- **4. Coding / Implementation** — translate design into source code in a programming language.
- **5. Testing** — verify and validate; find defects (unit → integration → system → acceptance).
- **6. Deployment** — release and install the product in the user environment.
- **7. Maintenance** — correct, adapt, perfect and prevent defects after delivery.

Memory Trick: Phase order = "**Please Read Design Carefully To Deliver Maintainably**" (Planning, Requirements, Design, Coding, Testing, Deployment, Maintenance).

MCQ Trap: **Maintenance** is the **longest** and most **expensive** phase of the entire life cycle (often **60–70%** of total cost), NOT coding.

2.2 Software Configuration Management (SCM)

Software Configuration Management (SCM) is the set of activities designed to **manage change** throughout the software life cycle, applied **across all phases**.

A **Software Configuration Item (SCI)** is any information/work-product created as part of the process (code, SRS, design docs, test cases, data).

Baselines

A **Baseline** (IEEE) is "a specification or product that has been **formally reviewed and agreed upon**, that thereafter serves as the basis for further development, and that **can be changed only through formal change-control procedures**."^{*}

Common baselines: **Functional / Requirements baseline**, **Design baseline**, **Product baseline**. An SCI becomes a baseline only **after** a formal technical review.

The Five SCM Tasks

#	SCM Task	Purpose
1	Identification	Uniquely name and track each SCI
2	Version Control	Manage versions/revisions using a repository
3	Change Control	Control how changes are requested and approved
4	Configuration Audit	Verify a change was done correctly and completely
5	Status Reporting	Record and report the status of SCIs and changes

Change Control Process

Change Request --> Evaluation (impact) --> Decision (CCA/CCB) -->
 ECO issued --> Check-OUT item --> Make change --> Review (SQA) -->
 Check-IN item --> Baseline updated

- **ECO = Engineering Change Order** (defines the change, constraints, review criteria).
- **CCB/CCA = Change Control Board / Authority** approves or rejects requests.
- **Check-out / Check-in** with **access control** and **synchronization control** prevent conflicting edits.

Version Control Concepts

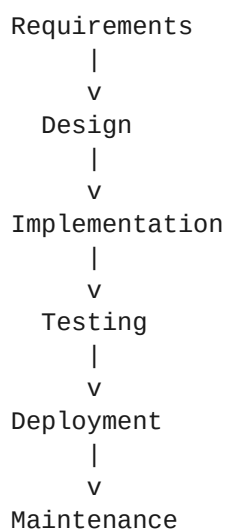
Version control combines a **version repository**, a **version-management capability** (versions, variants, revisions), and a **make facility**. A **variant** is a functionally equivalent alternate version; a **revision** is a change that supersedes the previous one.

MCQ Trap: Configuration Audit verifies the change was performed correctly; **Status Reporting (status accounting)** communicates *what changed, who did it, when, and what is affected*. Do not swap these two.

3. Software Development / Process Models

3.1 Waterfall Model (Classical Linear Sequential)

The **Waterfall Model** (Royce, 1970) is the oldest model where phases flow **downward** linearly; a phase begins only after the previous one **completes** and is reviewed.



Advantages: simple, easy to **understand and manage**, well-documented, works when requirements are **fixed and well-understood**, each phase has defined deliverables.

Disadvantages: **no working software** until late; **high risk**; cannot handle **changing requirements**; **customer** sees product only at the **end**; not suitable for large/complex/object-oriented projects.

Best for: small projects with **stable, clearly defined** requirements.

3.2 Iterative / Incremental Model

The **Incremental Model** delivers the product in a series of **increments (builds)**; the first increment is the **core product**, and each later increment **adds functionality**.

The **Iterative Model** develops a rough version first, then **refines** it through repeated cycles. Both combine elements of the waterfall applied **repeatedly**.

Advantage: working software is produced **early**; flexible; easier to **test and debug** smaller increments; lower delivery risk.

3.3 Prototyping Model

A **Prototype** is a **quick, partial, working model** built to clarify **unclear requirements** through user feedback.

Requirements gathering --> Quick design --> Build prototype -->
Customer evaluation --> Refine --> (loop) --> Engineer final product

Types: **Throwaway / Rapid prototype** (discarded after learning) and **Evolutionary prototype** (refined into the final product).

Advantage: reduces requirement **misunderstanding**; good when requirements are **fuzzy**.

Disadvantage: customer may mistake the **prototype** for the final product; quick design decisions may yield a **poor architecture**.

3.4 Spiral Model (Risk-Driven)

The **Spiral Model** (**Barry Boehm, 1988**) is a **risk-driven** meta-model that combines the **iterative** nature of prototyping with the **systematic** control of the waterfall. Each loop of the spiral is a **phase**, and **Risk Analysis** is its defining feature.

Quadrant 1	Quadrant 2
Determine objectives, alternatives, constraints	Identify & RESOLVE risks (risk analysis, prototypes)
Plan next iteration (customer evaluation)	Develop & verify next-level product
Quadrant 4	Quadrant 3

The **4 quadrants** of each spiral loop:

- **1. Determine objectives**, alternatives and constraints.
- **2. Evaluate alternatives; identify and resolve RISKS** (the key step; may build prototypes).
- **3. Develop and verify** the next-level product (engineering).
- **4. Plan** the next iteration (customer evaluation / review).

Advantages: explicit **risk management**; suitable for **large, complex, high-risk** projects; accommodates changes.

Disadvantages: **expensive**; requires **risk-assessment expertise**; not suitable for **small** projects; success depends heavily on the risk phase.

MCQ Trap: The defining/distinguishing feature of the **Spiral** model is **Risk Analysis**. Inventor = **Barry Boehm (1988)**.

3.5 RAD Model (Rapid Application Development)

RAD (proposed by **James Martin**) is an **incremental, component-based** model that achieves a **very short development cycle** (typically **60–90 days**) using **reusable components**, **CASE tools** and **multiple teams** working in parallel.

RAD phases: **Business Modeling** → **Data Modeling** → **Process Modeling** → **Application Generation** → **Testing & Turnover**.

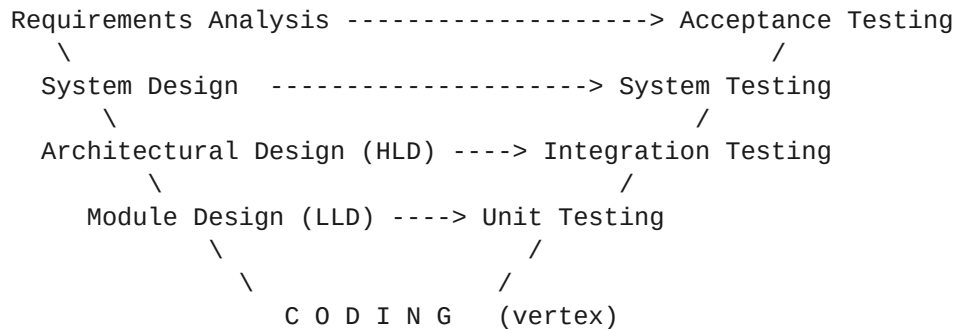
Requirements: **modular** system, **reusable** components, **scalable** & **highly skilled** developers, strong **customer involvement**.

Disadvantages: needs **highly skilled** developers and committed users; not suitable for **high-technical-risk** or non-modularizable systems; reduced scalability for very large unique projects.

Memory Trick: RAD = "**Build fast with reusable bricks in parallel**" → 60–90 days, component-based.

3.6 V-Model (Verification and Validation Model)

The **V-Model** is an extension of the waterfall in which **every development (left-side, Verification) phase has a corresponding testing (right-side, Validation) phase**, giving the model its **V** shape. Coding sits at the **vertex**.



Development (Verification) Phase	Corresponding Test (Validation) Phase
Requirements Analysis	Acceptance Testing
System Design	System Testing
Architectural (High-Level) Design	Integration Testing
Module (Low-Level) Design	Unit Testing

Advantages: testing planned early (in parallel with development); each phase has clear deliverables; high success for **well-defined** requirements.

Disadvantages: rigid, like waterfall; no early prototypes; poor for **changing** requirements.

MCQ Trap: In the V-Model, **Requirements** ↔ **Acceptance** testing and **HLD** ↔ **Integration** testing. A frequent trick swaps HLD with Unit testing — LLD (module design) maps to **Unit** testing.

3.7 Agile Model (Scrum, XP, Kanban)

Agile is an **iterative, incremental and adaptive** philosophy emphasising **working software, customer collaboration and responding to change**.

The **Agile Manifesto (2001)** states **4** core values (left valued more than right):

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

It is supported by **12 principles**. Popular Agile frameworks:

Scrum

Scrum organises work into time-boxed **Sprints** (usually **2–4 weeks**).

- **3 Roles:** **Product Owner** (owns backlog/priorities), **Scrum Master** (facilitator, removes impediments), **Development Team**.
- **3 Artifacts:** **Product Backlog**, **Sprint Backlog**, **Increment**.
- **Events:** **Sprint Planning**, **Daily Scrum (15-minute stand-up)**, **Sprint Review**, **Sprint Retrospective**.

Extreme Programming (XP)

XP emphasises engineering practices: **Pair Programming**, **Test-Driven Development (TDD)**, **Continuous Integration**, **Refactoring**, **Small Releases**, **Simple Design**, **On-site Customer**, **Collective Ownership**, **Coding Standards**. Its **4** activities are **Planning**, **Design**, **Coding**, **Testing**.

Kanban

Kanban visualises the workflow on a **Kanban board**, **limits Work-In-Progress (WIP)**, and uses a **continuous, pull-based flow** (no fixed-length iterations).

Memory Trick: Scrum stand-up = **15 minutes**; Sprint = **2–4 weeks**. XP's signature practice = **Pair Programming + TDD**. Kanban's signature = **WIP limits**.

3.8 Comparison and Selection of Models

Basis	Waterfall	Spiral	RAD	Agile
Approach	Linear	Iterative + Risk	Incremental/Component	Iterative/Adaptive
Risk Handling	Poor	Best (explicit)	Medium	Good
Requirement Changes	Not allowed	Accommodated	Accommodated	Welcomed
Customer Involvement	Only at ends	At each loop	High	Continuous
Best for	Small, stable	Large, high-risk	Short, modular	Dynamic, evolving
Working software	Late	Incremental	Fast	Early & continuous

Shortcut: Stable & small → **Waterfall**; high-risk & large → **Spiral**; tight deadline & modular → **RAD**; changing needs → **Agile**.

4. Requirement Engineering

4.1 Definition and Tasks

Requirement Engineering (RE) is the **systematic process** of **discovering, analysing, documenting, validating and managing** the requirements of a software system. It bridges the gap between **customer needs** and the **design**.

Pressman's 7 RE tasks (in order): **Inception → Elicitation → Elaboration → Negotiation → Specification → Validation → Requirements Management**.

Memory Trick: RE tasks = "I-E-E-N-S-V-M" → **"I Eat Every Nice Soft Veg Meal."**

4.2 Requirement Elicitation Techniques

Elicitation is the activity of **gathering** requirements from stakeholders. Major techniques:

- **Interviews** — open-ended or structured one-to-one questioning.
- **Questionnaires / Surveys** — collect data from many stakeholders cheaply.
- **Brainstorming** — group generation of ideas without early criticism.
- **Prototyping** — build a model to draw out requirements through feedback.
- **FAST (Facilitated Application Specification Technique)** — joint facilitated meeting of developers and customers.
- **QFD (Quality Function Deployment)** — maps customer value into technical requirements; classifies into **Normal, Expected, Exciting** requirements.
- **Use cases / Scenarios, Observation (ethnography), Document analysis.**

4.3 Types of Requirements: Functional vs Non-Functional

Basis	Functional Requirements (FR)	Non-Functional Requirements (NFR)
Defines	What the system does (functions, services)	How well the system performs (qualities)
Examples	Login, payment, report generation	Performance, security, usability, reliability, scalability
Also called	Behavioural requirements	Quality attributes / "-ilities"
Verification	Functional/black-box testing	Performance, load, security testing

Requirements are also classified as **User requirements** (high-level, customer language) vs **System requirements** (detailed, technical), and **Domain requirements** (from the application domain).

MCQ Trap: "The system shall respond within 2 seconds" is a **Non-Functional** (performance) requirement. "The system shall let users transfer funds" is **Functional**. Examiners love swapping these.

4.4 Requirement Analysis and Modeling

Requirement analysis refines elicited requirements, detects **conflicts/omissions**, and builds **analysis models** such as:

- **Data models** — Entity-Relationship Diagram (**ERD**).
- **Functional models** — Data Flow Diagram (**DFD**).
- **Behavioural models** — State Transition Diagram (**STD**).

4.5 Software Requirements Specification (SRS)

The **SRS** is the official document that **completely and precisely describes the behaviour** of the system to be developed; it is the **contract** between customer and developer. The relevant standard is **IEEE 830**.

Characteristics of a Good SRS (IEEE 830)

A good SRS must be:

- **Correct** — every stated requirement is one the software must meet.
- **Unambiguous** — every requirement has exactly **one** interpretation.
- **Complete** — all significant requirements are present.
- **Consistent** — no requirement conflicts with another.
- **Ranked** for **importance and/or stability**.
- **Verifiable** — a finite, cost-effective process exists to check it.
- **Modifiable** — structure allows easy, consistent changes.
- **Traceable** — origin is clear and it can be referenced forward/backward.

Memory Trick: SRS qualities = "**Correct Unambiguous Complete Consistent, Verifiable Modifiable Traceable Ranked**" → "**C-U-C-C-V-M-T-R**".

4.6 Requirement Validation and Management

Requirement Validation checks that the documented requirements are **correct, complete and consistent** before design. Techniques: **requirement reviews, prototyping, test-case generation, model validation, checklists**.

Requirements Management controls **changes** to requirements over the project's life (traceability matrices, change control).

4.7 Feasibility Study

A **Feasibility Study** assesses whether the proposed system is **worth building**. The **TELOS** dimensions:

Type	Question Answered
Technical	Can it be built with current technology/skills ?
Economic	Do benefits outweigh costs (Cost-Benefit Analysis, ROI)?
Legal	Does it satisfy laws/regulations (data, IP)?
Operational	Will it work within the organisation ?
Schedule	Can it be delivered on time ?

MCQ Trap: Economic feasibility uses **Cost-Benefit Analysis (CBA)** and is often the most decisive. The three "core" types tested are **Technical, Economic, Operational**.

5. Software Design

5.1 Design Concepts and Principles

Software Design transforms the **SRS (what)** into a blueprint for **construction (how)**. The fundamental design concepts (Pressman):

- **Abstraction** — focus on **essential** features, hide detail. Levels: **procedural abstraction** (named sequence of actions) and **data abstraction** (named collection of data).
- **Modularity** — divide software into **separately named, addressable modules** ("divide and conquer"). Effort decreases as modules increase, but **integration cost** rises — there is an **optimal number of modules** (minimum total cost).
- **Refinement (Stepwise Refinement)** — a **top-down** strategy (**Niklaus Wirth**) that **elaborates** detail progressively; it is **complementary** to abstraction.
- **Information Hiding** — each module hides its **internal design decisions** from others, exposing only an interface.
- **Architecture** — overall **structure** of components and their relationships.
- **Functional Independence** — achieved via **high cohesion** and **low coupling**.
- **Refactoring** — improving internal structure **without changing external behaviour**.

```

Total |           *   total cost = effort + integration
Cost  |   *           . *
      |   |   *   . *   <- OPTIMAL number of modules (min cost)
      |   |   * *
      |   |   * *
      |   |   * *
      |   |   *
      |   |   *   module (development) effort
      +----- Number of Modules

```

5.2 Cohesion (Intra-Module — HIGH is Good)

Cohesion measures the degree to which the elements **within a single module belong together** and contribute to a **single task**. **High cohesion is desirable**.

Types of Cohesion (worst → best)

Level	Type	Elements are grouped because...	Quality
1	Coincidental	... no meaningful relationship (arbitrary)	Worst
2	Logical	...they perform the same logical class of action (selected by a flag)	Low
3	Temporal	...they execute at the same time (e.g., init/startup)	Low
4	Procedural	...they follow a certain sequence of execution	Medium
5	Communication	...they operate on the same input/output data	Good
6	Sequential	... output of one is input to the next	Better
7	Functional	...all contribute to one single, well-defined task	Best

Memory Trick: Cohesion worst→best = **Co-Lo-Te-Pro-Co-Se-Fun** → *"**C**oincidentally **L**ogical **T**emporal **P**rocedures **C**ommunicate **S**equentially **F**unctionally."*

5.3 Coupling (Inter-Module — LOW is Good)

Coupling measures the degree of **interdependence between modules**. **Low (loose) coupling is desirable** because it improves maintainability and reduces ripple effects.

Types of Coupling (worst/tightest → best/loosest)

Level	Type	Description	Quality
1	Content	One module directly modifies/refers to internal data of another	Worst
2	Common	Modules share global data	High
3	External	Modules share an externally imposed format/protocol/device	High
4	Control	One module controls the logic of another via a flag/control	Medium
5	Stamp	Modules pass a whole data structure but use only part of it	Low
6	Data	Modules communicate only via simple parameters (scalar data)	Best

Memory Trick: Coupling worst→best = **Content, Common, External, Control, Stamp, Data** → "**Cars Can't Exist Containing Stale Diesel.**"

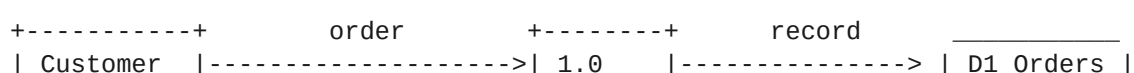
MCQ Trap: Goal of good design = **HIGH cohesion + LOW coupling**. **Content** coupling is worst; **Data** coupling is best. **Functional** cohesion is best; **Coincidental** is worst. Students frequently invert one of these.

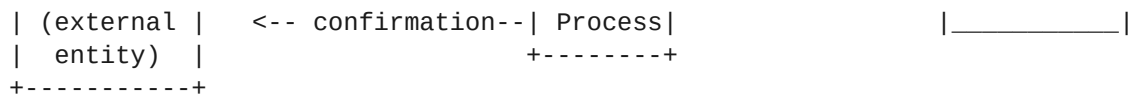
5.4 Design Approaches

- **Function-Oriented Design** — **top-down** decomposition into **functions/procedures**; data is shared. Tools: **DFD, Structure Chart, Data Dictionary**.
- **Object-Oriented Design (OOD)** — system modelled as interacting **objects/classes**; uses **encapsulation, inheritance, polymorphism, abstraction**; tends to be **bottom-up**.
- **Structured Design** (Yourdon & Constantine) — converts DFDs into **structure charts** via **transform** and **transaction** analysis.
- **User Interface (UI) Design** — golden rules: **Place the user in control, Reduce the user's memory load, Make the interface consistent**.

Data Flow Diagram (DFD)

A **DFD** (a.k.a. **bubble chart**) graphically shows how **data flows** through a system. **It shows data flow, NOT control flow, decisions, loops or timing.**





Symbols: []=external entity O/bubble=process open box(=)=data store arrow=data

- **Level-0 DFD = Context Diagram** — the **entire system as a single process** with external entities.
- **Level-1, Level-2 ...** — progressively **decompose** processes (this is called **levelling/explosion**).
- Rule of **balancing**: data flows into/out of a process must match its lower-level DFD.

Data Dictionary Notation

Symbol	Meaning
=	is composed of
+	AND (sequence)
[]	OR / select one of
{ }	iteration / repetition
()	optional
* *	comment

A **Structure Chart** shows the module hierarchy and **couples**: a **data couple** = arrow with an **empty circle**; a **control couple (flag)** = arrow with a **filled circle**.

6. Software Metrics and Project Management

6.1 Software Measurement and Metrics

A **measure** is a single quantitative figure; a **metric** relates measures (e.g., defects per KLOC); an **indicator** is a metric giving insight to managers.

Lines of Code (LOC)

LOC is a **size-oriented**, **direct** metric. **KLOC** = thousand LOC. Derived metrics:

- **Productivity = KLOC / Person-Month**
- **Quality = Defects / KLOC**
- **Cost = Total Cost / KLOC**

Disadvantage: language-dependent, penalises good high-level design, hard to estimate early, no functionality view.

Function Points (FP)

Function Points (Allan Albrecht, IBM, 1979) measure functionality from the **user's view**; they are **language-independent** and can be computed **early**.

FP uses **5** information-domain (function) types:

#	Function Type	Description
1	External Inputs (EI)	inputs that update internal files
2	External Outputs (EO)	reports/results sent out
3	External Inquiries (EQ)	input-output query pairs (no file update)
4	Internal Logical Files (ILF)	internal data maintained
5	External Interface Files (EIF)	data referenced from other systems

Formulas:

$$\begin{aligned}
 \text{UFP} &= \text{Sum of (count of each type } \times \text{ its weight)} && [\text{Unadjusted FP}] \\
 \text{CAF} &= 0.65 + 0.01 \times (\text{Sum of } F_i) && \text{where 14 factors } F_i \text{ each rated } 0..5 \\
 \text{FP} &= \text{UFP} \times \text{CAF}
 \end{aligned}$$

Since the **14** General System Characteristics give ΣFi in range **0–70**, the **CAF** lies between **0.65 and 1.35**.

MCQ Trap: FP information-domain count is **5** (EI, EO, EQ, ILF, EIF). The complexity factor count is **14**, each scored **0–5**. **FP = UFP × [0.65 + 0.01 × ΣFi]**.

6.2 COCOMO — Constructive Cost Model

COCOMO (Barry Boehm, 1981) estimates **effort, time and cost** from program size in **KLOC**. It defines **3 project modes**:

Mode	Nature	Size
Organic	small team, well-understood , stable	< 50 KLOC
Semi-Detached	medium, mixed experience	50–300 KLOC
Embedded	tight constraints , complex, real-time	> 300 KLOC

Basic COCOMO

$$\begin{aligned} \text{Effort } E &= a \times (\text{KLOC})^b && [\text{Person-Months, PM}] \\ \text{Time } D &= c \times (E)^d && [\text{Months}] \\ \text{Persons } P &= E / D \end{aligned}$$

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

The Three COCOMO Levels

- **Basic COCOMO** — rough estimate from **size (KLOC)** only.
- **Intermediate COCOMO** — multiplies by **EAF (Effort Adjustment Factor)** derived from **15 cost drivers**: $E = a \times (\text{KLOC})^b \times \text{EAF}$. Intermediate coefficients: Organic (**3.2, 1.05**), Semi-Detached (**3.0, 1.12**), Embedded (**2.8, 1.20**).
- **Detailed (Complete) COCOMO** — applies cost drivers **phase-by-phase** over a three-level product hierarchy.

The **15 cost drivers** fall in **4** groups: **Product, Hardware (Computer), Personnel, Project** attributes.

MCQ Trap: Embedded has the **highest** exponent **b = 1.20** (most effort growth); **Organic b = 1.05** is lowest. Note **Basic-Organic a = 2.4** but **Intermediate-Organic a = 3.2** — the constants differ between levels.

6.3 Project Scheduling (Gantt, PERT/CPM)

Gantt Chart

A **Gantt Chart** (Henry Gantt) is a **horizontal bar chart** plotting **tasks vs time**; it shows **start/end dates, durations and overlap/parallelism**, but is **weak at showing dependencies**.

PERT and CPM

Basis	PERT	CPM
Full form	Program Evaluation & Review Technique	Critical Path Method
Nature	Probabilistic	Deterministic
Orientation	Event-oriented	Activity-oriented
Time estimates	Three (optimistic, most-likely, pessimistic)	One
Best for	Research / uncertain projects	Repetitive / well-known projects
Origin	US Navy, Polaris, 1958	DuPont, 1957

PERT expected time: $t_e = (t_o + 4t_m + t_p) / 6$, where **to** = optimistic, **tm** = most-likely, **tp** = pessimistic. Variance $\sigma^2 = ((t_p - t_o)/6)^2$.

The **Critical Path** is the **longest** path through the network and equals the **shortest possible project duration**; its activities have **zero slack/float**. **Slack (float)** = time an activity can slip without delaying the project.

MCQ Trap: Critical path = LONGEST path in the network yet gives the **MINIMUM** completion time. PERT = probabilistic/**3** estimates; CPM = deterministic/**1** estimate.

6.4 Risk Management

Risk is a **potential future problem** (probability × impact). The **RMMM** plan covers **Risk Mitigation, Monitoring and Management**. Steps: **Identification** → **Projection (estimation)** → **Assessment** → **Mitigation/Monitoring**.

Risk categories: **Project risks** (schedule, budget, staff), **Technical risks** (design, implementation), **Business risks** (market, management). Also **Known, Predictable, Unpredictable** risks.

Risk Exposure (RE) = Probability of risk × Cost (impact) if it occurs.

6.5 Software Quality Assurance (SQA)

SQA is the set of **planned and systematic** activities ensuring software meets quality standards; it is **process-oriented** (prevention).

Basis	Quality Assurance (QA)	Quality Control (QC)
Focus	Process (prevent defects)	Product (detect defects)
Activity	Audits, reviews, standards	Testing, inspection
Goal	Prevention	Detection/Correction

7. Software Testing**7.1 Testing Fundamentals and Objectives**

Software Testing is the process of **executing a program with the intent of finding errors** (Glenford Myers). A **good test case** has a **high probability of finding an as-yet-undiscovered error**; a **successful test** is one that **uncovered such an error**.

Dijkstra's principle: "Testing can show the **presence** of defects, but **never their absence**."*

Verification vs Validation

Basis	Verification	Validation
Question	"Are we building the product right ?"	"Are we building the right product?"
Checks against	Specifications/design	User needs/requirements
Type	Static + dynamic (reviews, inspections)	Dynamic (actual execution)
Performed	Throughout (early)	At the end

Error, Fault, Failure, Bug

Human Error / Mistake --> Fault / Defect / Bug (in code) --> Failure (at run time)
 (developer makes it) (the flaw sits in software) (deviation user observe)

- **Error / Mistake** — a **human** action producing an incorrect result.
- **Fault / Defect / Bug** — the **flaw in the code** caused by the error.
- **Failure** — the **observed incorrect behaviour** when a fault executes.

A **Test Case** = inputs + execution conditions + **expected results**. A **Test Suite** = a **collection of test cases**.

MCQ Trap: A **fault** that is never executed never causes a **failure**. Order is **Error → Fault → Failure**.
 "Bug" = "Fault" = "Defect."

7.2 Levels of Testing (Order)

Unit --> Integration --> System --> Acceptance (+ Regression after any change)
 (module) (interfaces) (whole) (customer)

- **1. Unit Testing** — tests the **smallest unit/module** in isolation; done by the **developer**; uses **stubs & drivers**; mostly **white-box**.
- **2. Integration Testing** — tests **interfaces** between combined modules.
- **3. System Testing** — tests the **complete integrated system** against the SRS; **black-box**; includes **recovery, security, stress, performance** testing.
- **4. Acceptance Testing** — done by the **customer/user** to validate against requirements; includes **Alpha & Beta, UAT**.
- **Regression Testing** — **re-running** existing tests after a change to ensure no **new defects** were introduced.

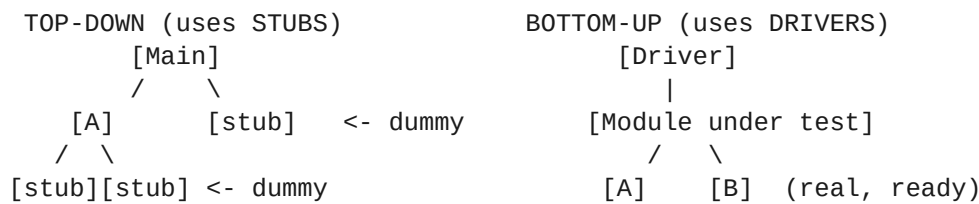
MCQ Trap: Order is **Unit → Integration → System → Acceptance**. **Regression** is not a level — it is performed whenever code changes (during maintenance especially).

7.3 Integration Testing Strategies — Stubs vs Drivers

- **Big-Bang** — all modules combined **at once**; hard to **localise** faults.
- **Top-Down** — integrate from the **top (main) module downward**; replaces **not-yet-ready lower modules** with **STUBS**. Can be **depth-first** or **breadth-first**.
- **Bottom-Up** — integrate from the **lowest modules upward**; replaces **not-yet-ready calling modules** with **DRIVERS**.
- **Sandwich / Hybrid** — combines **top-down + bottom-up** around a central **target layer**.

Aspect	Stub	Driver
What it is	Dummy called (sub-ordinate) module	Dummy calling (super-ordinate) module
Replaces	Lower-level module not yet ready	Higher-level module not yet ready
Used in	Top-Down integration	Bottom-Up integration

Aspect	Stub	Driver
Direction	Called by the module under test	Calls the module under test



Memory Trick: "Stub → Subordinate (called) → Top-down"; "Driver → Drives down / calls → Bottom-up." Or: "**Stubs come down (Top-down), Drivers push up (Bottom-up).**"

MCQ Trap: Top-down ⇒ **Stubs**; Bottom-up ⇒ **Drivers**. A stub is **called by** the tested module; a driver **calls** the tested module. Swapping these is the single most common testing MCQ trick.

7.4 White-Box (Structural) Testing

White-Box Testing (a.k.a. **Glass-box, Clear-box, Structural**) designs tests from the **internal code structure/logic**. Coverage criteria (increasing strength):

- **Statement Coverage** — every **statement** executed at least once (weakest).
- **Branch / Decision Coverage** — every **branch (true & false)** of each decision taken.
- **Condition Coverage** — every **boolean sub-condition** evaluated both ways.
- **Path Coverage** — every **independent path** executed (strongest, costliest).

Branch coverage subsumes statement coverage; **path coverage** is the most thorough.

Cyclomatic Complexity (McCabe, 1976)

Cyclomatic Complexity V(G) measures the number of **linearly independent paths** through a program's control-flow graph; it also equals the **minimum number of test cases** for basis-path coverage.

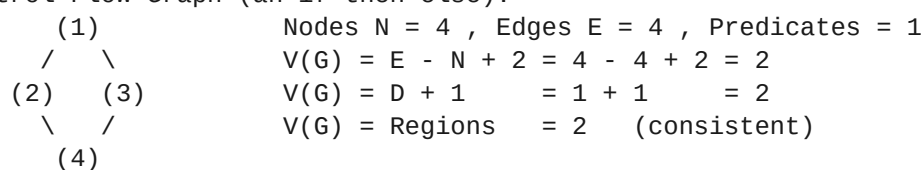
Three equivalent formulas (single connected program, P = components):

$$V(G) = E - N + 2P \quad (E = \text{edges}, N = \text{nodes}; \text{for one program } P=1 \rightarrow E-N+2)$$

$$V(G) = D + 1 \quad (D = \text{number of decision/predicate nodes})$$

$$V(G) = \text{Number of REGIONS in the planar flow graph (incl. outer region)}$$

Control-Flow Graph (an if-then-else):



McCabe recommended keeping **V(G) ≤ 10** per module. **Basis Path Testing** uses V(G) to derive the **basis set** of independent test paths.

Control-flow testing is based on the flow graph; **Data-flow testing** is based on **definition–use (def-use)** chains of variables.

MCQ Trap: The most-tested formula is **V(G) = E – N + 2P** (or **E – N + 2** for one component). Equivalently **V(G) = Predicate nodes + 1 = number of regions**. All three must give the **same** value.

7.5 Black-Box (Functional) Testing

Black-Box Testing (a.k.a. **Functional, Behavioural**) designs tests **only from requirements/specs**, with **no knowledge of internal code**. Techniques:

- **Equivalence Partitioning (ECP)** — divide inputs into **valid & invalid classes**; test **one representative** per class.

- **Boundary Value Analysis (BVA)** — test at the **edges** of ranges, where defects cluster. For a range $[a, b]$, test $a-1, a, a+1 \dots b-1, b, b+1$.
- **Decision Table Testing** — tabulate **combinations of conditions** → **actions**.
- **Cause-Effect Graphing** — graph mapping **causes (inputs)** to **effects (outputs)**; converted into a decision table.
- **State Transition Testing** and **Error Guessing**.

Test Data Suite Preparation

A **test data suite** is the planned set of **inputs and expected outputs** built from the above techniques to maximise defect detection with minimal cases.

Alpha vs Beta Testing

Basis	Alpha Testing	Beta Testing
Location	Developer's site	Customer/User's site
Environment	Controlled	Real / uncontrolled
Developer present?	Yes	No
Stage	Before beta	After alpha, before release

Both **Alpha** and **Beta** are forms of **Acceptance Testing**.

MCQ Trap: BVA catches **off-by-one / boundary** errors; **Equivalence Partitioning** reduces the **number** of test cases. **Alpha** = developer's site; **Beta** = customer's site.

7.6 Static Testing Strategies (No Code Execution)

Static testing examines work-products **without executing code** (a **verification** activity). It finds defects **early and cheaply**.

- **Formal Technical Review (FTR) / Peer Review** — the umbrella term; includes walkthroughs and inspections; focuses on **uncovering defects** in a work product.
- **Walkthrough** — **informal, led by the author**, who guides the team through the product; few formal rules; no follow-up required.
- **Inspection (Fagan Inspection)** — the **most formal** review; **led by a trained moderator (NOT the author)**; uses **checklists**, defined roles, and mandatory **follow-up**.
- **Code Inspection** — inspection applied to source code against a defect checklist.
- **Compliance with Design and Coding Standards** — checking the product conforms to agreed conventions/standards.

Roles in a Formal Inspection

Role	Responsibility
Moderator	Leads/controls the inspection (not the author)
Author / Producer	Created the work product; clarifies, fixes
Reader	Paraphrases / presents the product
Recorder (Scribe)	Logs the defects found
Inspector / Reviewer	Finds defects

MCQ Trap: A **Walkthrough** is **author-led and informal**; an **Inspection** is **moderator-led, formal, checklist-based** with follow-up. Both are **static** (no execution).

8. Software Maintenance

8.1 Need and Process

Software Maintenance is the **modification of a software product after delivery** to correct faults, improve performance, or adapt to a changed environment. It is the **longest and costliest** phase (often **60–70%** of total life-cycle cost).

8.2 Types of Maintenance (4)

Type	Trigger / Purpose	Nature	Approx. Share*
Corrective	Fix bugs/faults found after release	Reactive	~21%
Adaptive	Adapt to changed environment (OS, hardware, law)	Reactive	~25%
Perfective	Add/enhance features, improve performance (new user needs)	Proactive	~50% (largest)
Preventive	Improve future maintainability , prevent latent faults	Proactive	~4%

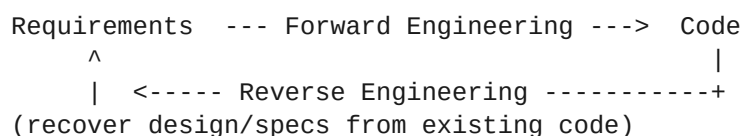
(*Lientz & Swanson study; values are approximate and commonly tested.)

Memory Trick: Maintenance types = "**CAPP**" → **C**orrective, **A**daptive, **P**erfective, **P**reventive. **Perfective is the largest; Preventive is the smallest.**

MCQ Trap: Fixing a **Y2K / new-OS** issue = **Adaptive** (environment changed). Fixing a **reported bug** = **Corrective**. Adding a **new feature** = **Perfective**. Re-structuring to ease future change = **Preventive**.

8.3 Re-engineering and Reverse Engineering

- **Forward Engineering** — the normal flow: **requirements → design → code**.
- **Reverse Engineering** — **analysing existing code to recover its design/specification** ("design recovery"); moves **backward** from code to a higher abstraction. It is a process of **examination, not change**.
- **Software Re-engineering** — **examining and altering** a legacy system to **reconstitute it in a new form** (restructuring + re-documentation), then re-implementing. Steps: **Inventory analysis → Document restructuring → Reverse engineering → Code & data restructuring → Forward engineering**.



MCQ Trap: **Reverse engineering** = code → design (recovery, no change). **Re-engineering** = reverse engineering **plus** forward engineering to produce an **improved** system.

8.4 Software Reliability and Availability

Software Reliability is the **probability of failure-free operation** of a program for a **specified time** in a **specified environment**. Key metrics:

- **MTTF = Mean Time To Failure** (average uptime before a failure).
- **MTTR = Mean Time To Repair** (average time to fix).
- **MTBF = MTTF + MTTR** (Mean Time Between Failures).
- **Availability** = $[\text{MTTF} / (\text{MTTF} + \text{MTTR})] \times 100\%$ = uptime ÷ total time.

Lehman's Laws of software evolution include **Continuing Change** (a system must adapt or become less useful) and **Increasing Complexity** (entropy rises unless work is done to reduce it).

MCQ Trap: **MTBF = MTTF + MTTR** is the most-tested relationship. **Reliability** ↑ as **MTTF** ↑; **Availability** depends on **both** MTTF and MTTR.

9. Quick Revision / Exam Pointers

- **SDLC order:** Planning → Requirements → Design → Coding → Testing → Deployment → Maintenance.
- **Models:** Waterfall = linear/stable; Spiral = **risk-driven (Boehm)**; RAD = **fast, component-based (60–90 days)**; V-Model = **verification ↔ validation**; Agile = **adaptive**.
- **Cohesion (HIGH good):** Coincidental < Logical < Temporal < Procedural < Communicational < Sequential < **Functional**.
- **Coupling (LOW good):** Data < Stamp < Control < External < Common < **Content (worst)**.
- **Metrics:** FP info-domain = **5**, factors = **14**, **CAF = 0.65 + 0.01ΣFi**; COCOMO modes = **Organic/Semi/Embedded**, Effort $E = a(KLOC)^b$.
- **Testing levels:** Unit → Integration → System → Acceptance (+ Regression).
- **Stubs = Top-Down; Drivers = Bottom-Up.**
- **Cyclomatic Complexity:** $V(G) = E - N + 2P = D + 1 = \text{Regions}$.
- **Static reviews:** Walkthrough (informal, author-led) vs Inspection (formal, moderator-led).
- **Maintenance (CAPP):** Corrective, Adaptive, Perfective (largest), Preventive (smallest).
- **Reliability:** **MTBF = MTTF + MTTR.**